

Internet Indirection Infrastructure

Ion Stoica¹, Daniel Adkins¹, Sylvia Ratnasamy¹, Scott Shenker²,
Sonesh Surana¹, and Shelley Zhuang¹

¹ University of California at Berkeley, Berkeley CA 94720, USA,
{istoica, dadkins, sylvia, sonesh, shelleyz}@cs.berkeley.edu,

² The ICSI Center for Internet Research, Berkeley CA 94704, USA,
shenker@cs.icsi.berkeley.edu

Abstract. This paper argues for an Internet Indirection Infrastructure that replaces the point-to-point communication abstraction of today's Internet with a rendezvous-based communication abstraction: instead of explicitly sending a packet to a destination, each packet is associated an identifier, which is then used by the receiver to get the packet. This level of indirection decouples the sender and the receiver behaviors, and allows us to efficiently support basic communication services such as multicast, anycast and mobility in the Internet. To demonstrate the feasibility of this approach, we are currently designing and building an overlay network solution based on the Chord lookup system.

1 Introduction

The Internet is designed around the *unicast* point-to-point communication abstraction. This simple abstraction is one of the main reasons behind its scalability and efficiency. However, as the Internet evolves into a global communication infrastructure, there is an increasing need to support other communication abstractions such as multicast, anycast, and host mobility. Unfortunately, despite years of intense research, these services are yet to be deployed in the Internet. The main difficulty resides in the fact that the point-to-point communication abstraction—which assumes one sender and one receiver placed at well-known and fixed network locations—is *not* appropriate for these services. For example, mobility requires one to remove the assumption that end-hosts are fixed, multicast requires one to remove the assumption that there is only one sender and one receiver, and anycast requires one to remove the assumption that the receiver's location is known.

To get around this problem, existing solutions use a simple but powerful technique: *indirection*. These solutions assume a physical or a logical indirection point interposed between the sender and the receiver(s) that relays the traffic between them. By communicating through the indirection point rather than directly to the end-host, a sender can abstract away the location and the number of receivers. For instance, mobile IP assumes a home agent that hides the end-host mobility, while IP multicast assumes a logical indirection point (address) that hides the number of receivers and their locations.

While indirection can enable these services, implementing them at the IP layer in a scalable fashion has proven difficult [4, 9, 14]. Moreover, deploying additional functionality at the IP layer requires a level of community-wide consensus and commitment that is hard to achieve. In short, implementing these more general abstractions at the IP layer poses difficult technical problems and formidable deployment barriers.

In response, many researchers have turned to application-layer solutions (either end-host or overlay mechanisms) to support these abstractions [2, 4, 11, 13]. While these proposals achieve the desired functionality, they do so in a piecemeal fashion; *e.g.*, solutions for mobility do not address multicast, and vice versa. As a result, many similar and largely redundant mechanisms are implemented to achieve these various goals.

The goal of this work is to combine the generality of IP-layer solutions with the deployability of overlay solutions by proposing a general-purpose Internet indirection infrastructure (*i3*). This infrastructure supports indirection by providing a flexible *rendezvous*-based communication abstraction. A variety of communication services, such as multicast, anycast, and mobility, can be easily implemented on top of this communication abstraction. Such an approach will avoid both the technical and deployment challenges inherent in IP-layer solutions and the redundancy and lack of synergy in application-layer approaches.

2 Internet Indirection Infrastructure

The purpose of the Internet Indirection Infrastructure (*i3*) is to provide indirection; that is, it decouples the act of sending from the act of receiving. The *i3* service model is simple: sources send packets to a logical *identifier*, and receivers express interest in packets sent to an identifier.

This service model is similar to that of IP multicast. The crucial difference is that the *i3* equivalent of an IP multicast join is more flexible. IP multicast offers a receiver a binary decision of whether or not to receive packets sent to that group (this can be indicated on a per-source basis). It is up to the multicast infrastructure to build efficient delivery trees. The *i3* equivalent of a join is inserting a *trigger*. It allows receivers to *control* the routing of the packet. This provides two advantages. First, it allows them to create, at the application level, services such as mobility, anycast, and service composition out of this basic service model. Thus, this one simple service model can be used to support a wide variety of application-level communication abstractions, alleviating the need for many parallel and redundant overlay infrastructures. Second, the infrastructure can give responsibility for efficient tree construction to the end-hosts. This allows the infrastructure to remain simple, robust, and scalable.

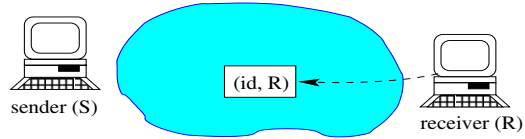
2.1 Rendezvous-Based Communication

The service model is instantiated as a rendezvous-based communication abstraction. In their simplest form, packets are pairs $(id, data)$ where *id* is an *m*-bit

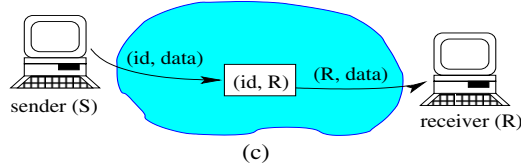
identifier¹ and *data* is the payload (typically a normal IP packet payload). Receivers use *triggers* to indicate their interest in packets. In their simplest form, triggers are pairs $(id, addr)$, where *id* is the trigger identifier, and *addr* is a node's address, consisting of an IP address and UDP port number. A trigger $(id, addr)$ indicates that all packets sent to identifier *id* should be forwarded (at the IP layer) by the *i3* infrastructure to the node with address *addr*. More specifically, the rendezvous-based communication abstraction exports the three primitives shown in Figure 1(a).

<i>i3</i> 's Application Programming Interface (API)	
<i>sendPacket(p)</i>	send packet
<i>insertTrigger(t)</i>	insert trigger
<i>removeTrigger(t)</i>	remove trigger

(a)



(b)



(c)

Fig. 1. (a) *i3*'s API. Example illustrating communication between two nodes: (b) The receiver *R* inserts trigger (id, R) . (c) The sender sends packet $(id, data)$.

Figure 1(b) illustrates the communication between two nodes, where receiver *R* wants to receive packets sent to *id*. The receiver inserts the trigger (id, R) into the network. When a packet is sent to identifier *id*, the trigger causes it to be forwarded via IP to *R*. There are two generalization that extend the power and the flexibility of the rendezvous-based communication abstraction. The first generalization allows *inexact* matching between identifiers, and it is used to provide anycast functionality. The second generalization replaces identifiers with a stack of identifiers and is used to provide service composition (see Section 2.3).

¹ In our design we use $m = 256$. Such a large value of m allows end hosts to choose trigger identifiers independently since the chance of collision is minimal.

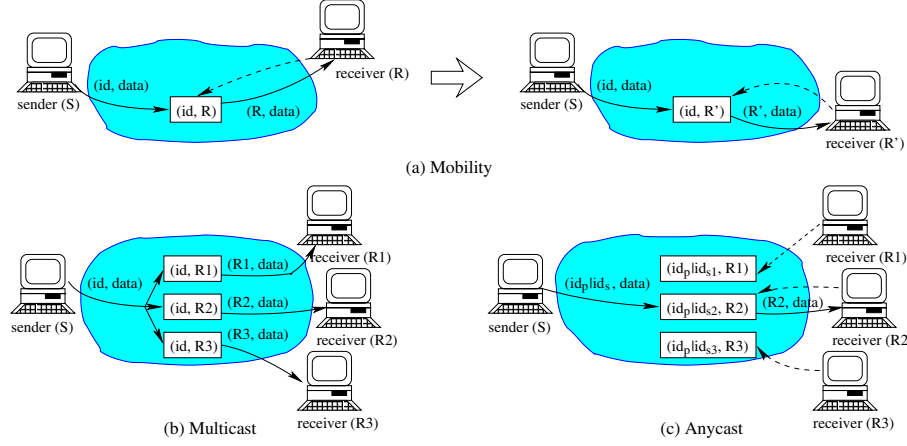


Fig. 2. Communication abstractions provided by *i3*. (a) Mobility: The change of the receiver's address from R to R' is transparent to the sender. (b) Multicast: Every packet $(id, data)$ is forwarded to each receiver R_i that inserts the trigger (id, R_i) . (c) Anycast: The packet matches the trigger of receiver $R2$. $id_p|id_s$ denotes an identifier of size m , where id_p represents the prefix of the k most significant bits, and id_s represents the suffix of the $m - k$ least significant bits.

2.2 Communication Primitives Provided by *i3*

Next, we briefly discuss how *i3* can achieve the more general communication abstractions of mobility, multicast, and anycast.

Mobility: A mobile host that changes its address from R to R' as a result of moving from one subnetwork to another can preserve the end-to-end connectivity by simply updating each of its existing triggers from (id, R) to (id, R') , as shown in Figure 2(a). The sending hosts need not be aware of the mobile host's current location or address.

Multicast: Creating a multicast group is equivalent to having all members of the group register triggers with the same identifier id . As a result, any packet that matches id is forwarded to all members of the group (see Figure 1(b)). Note that unlike IP multicast, with *i3* there is no difference between unicast or multicast packets, in either sending or receiving. Such an interface gives maximum flexibility to the application.

Anycast: Anycast ensures that a packet is delivered to at most one receiver in a group. To achieve this, we generalize the exact matching in *i3* to a longest prefix matching in which the first k bits are required to always match. All hosts in an anycast group maintain triggers which are identical in the k most significant bits. These k bits play the role of the anycast group identifier. To send a packet to an anycast group, a sender uses an identifier whose k -bit prefix matches the anycast group identifier. The packet is then delivered to the member of the

group whose trigger identifier best matches the packet identifier according to the longest prefix matching rule (see Figure 1(c)).

2.3 Stack of Identifiers

A second generalization of *i3* replaces identifiers with identifier *stacks*. An identifier stack is a tuple of identifiers that takes the form $(id_1, id_2, id_3, \dots, id_k)$ where id_i is either an identifier or an address. Packets p and triggers t are thus of the form: $p = (id_{stack}, data)$, and $t = (id, id_{stack})$, respectively. The generalized form of packets allows a source to send a packet to a series of identifiers, much as in source routing. The generalized form of triggers allows a trigger to send a packet to another identifier rather than to an address.

To illustrate the use of the stack of identifiers, consider the problem of sending an MPEG video stream to one H.263 receiver and one MPEG receiver (see Figure 3). To provide this functionality, we use the ability of the *receiver*, to control the transformations performed on data packets. In particular, the H.263 receiver $R1$ inserts trigger $(id, (id_{MPEG-H.263}, R1))$, and the sender sends packets $(id, data)$. Each packet matches $R1$'s trigger, and as a result the packet's identifier id is replaced by the trigger's stack $(id_{MPEG-H.263}, T)$. Next, the packet is forwarded to the MPEG-H.263 transcoder, and then directly to receiver $R1$. In contrast, an MPEG receiver $R2$ only needs to maintain a trigger $(id, R1)$ in *i3*. This way, receivers with different display capabilities can subscribe to the same multicast group.

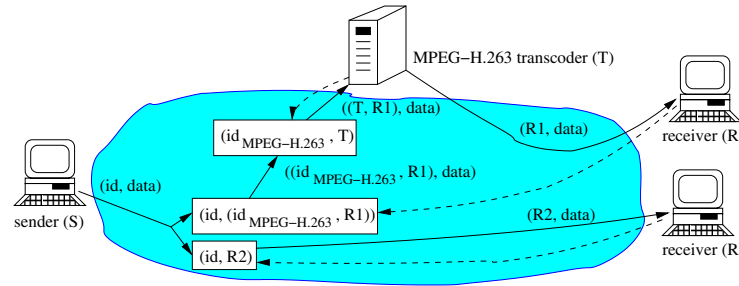


Fig. 3. Heterogeneous multicast: Receiver $R1$ specifies that wants to receive JPEG data, while $R2$ specifies that wants to receive MPEG data. The sender sends MPEG data.

3 *i3* Implementation

i3 is implemented as an overlay network which consists of a set of servers that store triggers and forward packets (using IP) between *i3* nodes and to end hosts.

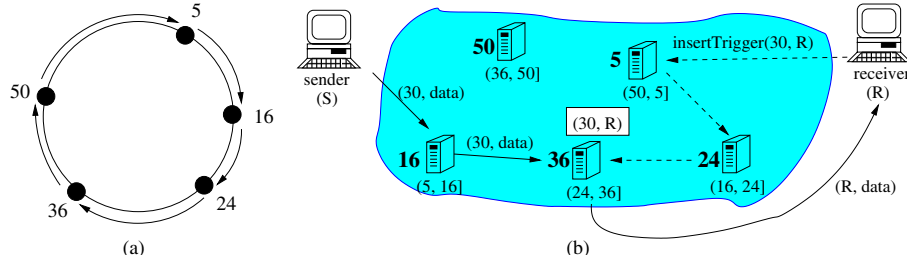


Fig. 4. (a) A Chord identifier circle for $m = 6$, with 5 servers identified by 5, 16, 24, 36, and 50, respectively. Each server is responsible for all identifiers between its identifier and the identifier of the node that precedes it on the circle. (b) Receiver R inserts trigger $(30, R)$, and the trigger is forwarded via *i3* to server 36 which is responsible for identifier 30. The trigger is stored there (shown in the white box) until explicitly removed or timed out. When sender S sends packet $(30, \text{data})$, it is also forwarded via *i3* to server 36. Servers identifiers are in bold. The interval of identifiers for which each server is responsible are also shown.

To maintain this overlay network and to route packets in *i3*, we use the Chord lookup protocol [5]. Chord assumes a circular identifier space of integers $[0, 2^m)$, where 0 follows $2^m - 1$. Every *i3* server has an identifier in this space, and all trigger identifiers belong to the same identifier space. The *i3* server with identifier n is responsible for all identifiers in the interval $(n_p, n]$, where n_p is the identifier of the node preceding n on the identifier circle. Figure 4(a) shows an identifier circle for $m = 6$. There are five *i3* servers in the system with identifiers 5, 16, 24, 36, and 50, respectively. All identifiers in the range $(5, 16]$ are mapped on server 16, identifiers in $(17, 24]$ are mapped on server 24, and so on.

When a trigger $(id, addr)$ is inserted, it is stored on the *i3* node responsible for id . When a packet is sent to id it is routed by *i3* to the node responsible for its id ; there it is matched against (any) triggers for that id and forwarded (using IP) to all hosts interested in packets sent to that identifier. Chord ensures that the server responsible for a identifier is found after visiting at most $O(\log n)$ other *i3* servers irrespective of the starting server (n represents the total number of servers in the system). To achieve this, Chord requires each node to maintain only $O(\log n)$ routing state. Chord allows servers to leave and join dynamically, and it is highly robust against failures. For more details refer to [5]. Figure 4(b) shows an example in which trigger $(30, R)$ is inserted at node 36 (i.e., the node that maps $(24, 36]$, and thus is responsible for identifier 30). Packet $(30, \text{data})$ is forwarded to server 30, matched against trigger $(30, R)$, and then forwarded via IP to R .

Note that packets are not stored in *i3*; they are only forwarded. *i3* provides a best-effort service like today's Internet. *i3* implements neither reliability nor ordered delivery on top of IP. End hosts use periodic refreshing to maintain their triggers in *i3*. Hosts contact an *i3* node when sending *i3* packets or inserting triggers. This *i3* node then forwards these packets or triggers to the *i3* node

responsible for the associated identifiers. Hosts need only know one *i3* node to use the *i3* infrastructure. This can be done through a static configuration file, or by a DNS lookup assuming *i3* is associated with a DNS domain name. In Figure 4(b), the sender knows only server 16, and the receiver knows only server 5.

One important observation is that once the end host finds the server that is responsible for a specific identifier, it can cache that server. As a result the end host can refresh the triggers and send packets with the same identifiers directly to that server. This significantly increases routing efficiency.

4 Security

Unlike IP, where an end-host can only send and receive packets, in *i3* end-hosts are also responsible for maintaining the routing information through triggers. While this allows flexibility for applications, it also (and unfortunately) creates new opportunities for malicious users. We now discuss several security issues and how *i3* addresses them.

We emphasize that our main goal here is not to design a bullet proof system. Instead, our goal is to design simple and efficient solutions that make *i3* not worse and in many cases better than today's Internet. The solutions outlined in this section should be viewed as a starting point towards more sophisticated and better security solutions that we will develop in the future.

An important component for using *i3* securely is for applications to make a distinction between two types of triggers: *public* and *private*. This distinction is made only at the application level: *i3* itself doesn't differentiate between private and public triggers.

The identifiers of a public trigger is known by all end-hosts in the system. An example is a web server that maintains a public trigger to allow any client to contact it. A public trigger can be defined as a the hash of the host's DNS name, of a web address, or of the public key associated to a web server. Public triggers are long lived, typically days or months. In contrast, private triggers are chosen by a small number of end-hosts and they are short lived. Typically, private triggers exist only during the duration of a flow. To illustrate the difference between public and private triggers, consider a client accessing a web server. First, the client chooses a private trigger identifier id_c , inserts trigger $(id_s, addr_c)$ into *i3*, and sends id_c to the web server via the server's public trigger. Once contacted, the server selects a private identifier id_s , inserts trigger $(id_s, addr_s)$, and sends id_s to the client via the client's trigger identifier id_c . The client and the server then use the private triggers to communicate. Once the communication terminates, the private triggers are destroyed.

Eavesdropping A user that knows a host's trigger can eavesdrop the traffic towards that host by inserting a trigger with the same identifier and its own address. We consider two cases: (a) private and (b) public triggers.

Private triggers are secretly chosen by the application end-points and are not supposed to be revealed to the outside world. The length of the trigger's

identifier makes it very difficult for a third party to use a brute force attack.² Furthermore, end-points can periodically change the private triggers associated with a flow. Another alternative would be for the receiver to associate multiple private triggers to the same flow, and the sender to send packets randomly to one of these private triggers. The alternative left to a malicious user is to intercept all private triggers. However this is equivalent to eavesdropping at the IP level or taking control of the *i3* server storing the trigger, which makes *i3* no worse than IP.

With *i3*, a public trigger is known by all users in the system, and thus anyone can eavesdrop the traffic to such a trigger. To alleviate this problem, end-hosts can use the public triggers to choose a pair of private triggers, and then use these private triggers to exchange the actual data. To keep the private triggers secret, one can use public key cryptography. To initiate a connection, a host *A* chooses a private trigger id_a , encrypts it under the public key of a receiver *B*, and then sends it to *B* via *B*'s public trigger. *B* decrypts *A*'s private trigger id_a , then chooses its own private trigger id_b , and sends this trigger back to *A* over *A*'s private trigger id_a . Since the sender's trigger is encrypted, a malicious user cannot impersonate *B*.³

Trigger hijacking A malicious user can isolate a host by removing its public trigger. Similarly, a malicious user in a multicast group can remove other members from the group by deleting their triggers. While removing a trigger also requires to specify the IP address of the trigger, this address is, in general, not hard to obtain.

One possibility to guard against this attack is to add another level of indirection. Consider a server *S* that wants to advertise a public trigger with identifier id_p . Instead of inserting the trigger (id_p, S) , the server can insert two triggers, (id_p, x) and (x, S) , where x is an identifier known only by *S*. Since a malicious user has to know x in order to remove either of the two triggers, this simple technique provides effective protection against this type of attack. To avoid performance penalties, the receiver can choose x such that both (id_p, x) and (x, S) are stored at the same server. With the current implementation this can be easily achieved by having i_p and x share the same k -bit prefix.

DoS Attacks The fact that *i3* gives end-hosts control on routing opens new possibilities for DoS attacks. We consider two types of attacks: (a) attacks on end-hosts, and (b) attacks on the infrastructure. In the former case, a malicious user can insert a hierarchy of triggers in which all triggers on the last level point

² While other application constraints such as storing a trigger at a server nearby can limit the identifier choice, the identifier is long enough (i.e., 256 bits), such that the application can always reserve a reasonable large number of bits (e.g., 64 bits) that are randomly chosen.

³ Note that an attacker can still count the number of connection requests to *B*. However, this information is of very limited use, if any, to the attacker. If, in the future, it turns out that this is a problem, then other security mechanisms such as public trigger authentication will need to be used.

to the victim. Sending a single packet to the trigger at the root of the hierarchy will cause the packet to be replicated and all replicas to be sent to the victim. This way an attacker can mount a large scale DoS attack by simply leveraging the *i3* infrastructure. In the later case, a malicious user can create trigger loops, for instance by connecting the leaves of a trigger hierarchy to its root. In this case, each packet sent to the root will be exponentially replicated!

To alleviate these attacks, *i3* uses three techniques:

1. **Challenges** *i3* assumes implicitly that a trigger that points to an end-host R is inserted by the end-host itself. An *i3* server can easily verify this assumption by sending a challenge to R the first time the trigger is inserted. The challenge consists of a random nonce that is expected to be returned by the receiver. If the receiver fails to answer the challenge the trigger is removed. As a result an attacker cannot use a hierarchy of triggers to mount a DoS attack (as described above), since the leaf triggers will be removed as soon as the server detects that the victim hasn't inserted them.
2. **Resource allocation** Each server uses Fair Queueing [7] to allocate resources amongst the triggers it stores. This way the damage inflicted by an attacker is only proportional to the number of triggers it maintains. An attacker cannot simply use a hierarchy of triggers with loops to exponentially increase its traffic. As soon as each trigger reaches its fair share the excess packets will be dropped. While this technique doesn't solve the problem, it gives *i3* time to detect and to eventually break the cycles.
To increase protection, each server can also put a bound on the number of triggers that can be inserted by a particular end-host. This will preclude a malicious end-host from monopolizing a server's resources.
3. **Loop detection** When a trigger that doesn't point to an IP address is inserted, the server runs a procedure to detect whether the new trigger doesn't create a loop. A simple procedure is to send a special packet with a random nonce. If the packet returns back to the server, the trigger is simply removed. To increase the robustness, the server can invoke this procedure periodically after such a trigger is inserted.

4.1 Anonymity

Point-to-point communication networks such as the Internet provide limited support for anonymity. Packets usually carry the destination and the source addresses, which makes it relatively easy for an eavesdropper to learn the sender and the receiver identities. In contrast, with *i3*, eavesdropping the traffic of a sender will not reveal the identity of the receiver, and eavesdropping the traffic of a receiver will not reveal the sender's identity. The level of anonymity can be further enhanced by using chain of triggers or stack of identifiers to route packets.

5 Related Work

The rendezvous-based communication is similar in spirit to the tuple space work in distributed systems [1, 10, 18]. However, tuple spaces have richer semantics, and they guarantee persistence and atomicity. Providing these properties in very large scale distributed systems is, however, very difficult. In contrast, *i3* trades these properties for a scalable and efficient implementation.

i3 shares many similarities with naming systems. This should come as no surprise, as identifiers can be viewed as semantic-less names. DNS maps hostnames to IP addresses [12]. DNS names are hierarchical while *i3* identifiers are flat. DNS resolvers form a static overlay hierarchy, while *i3* servers form a self-organizing overlay. *i3* integrates identifier resolution with packet forwarding. Active Names (AN) maps a name to a chain of mobile code responsible for locating the remote service [16]. While AN names are used primary to describe services, *i3* identifiers are used primary to abstract away the end-host location. Intentional Naming System (INS) is a resource discovery and service location system for mobile hosts [17]. *i3* differs from INS in that from network's point of view an identifier does not carry any semantic. Another difference is that *i3* allows end-hosts to explicitly control (via triggers) the application-level path followed by the packets.

The rendezvous-based abstraction is similar to the IP multicast abstraction [6]. An IP multicast address identifies the receivers of a multicast group in the same way an *i3* identifier identifies the multicast receivers. However, unlike IP which allocates a special range of addresses (i.e., class D) to multicast, *i3* does not put any restrictions on the identifier format. In addition, *i3* has ability to support multicast groups with heterogeneous receivers.

TRIAD [3] and IPNL [8] have been recently proposed to solve the IPv4 address scarcity problem. Both schemes use DNS names rather than addresses for global identification. One difference between *i3* and both TRIAD and IPNL is that the path of a packet is determined by end-hosts, instead of being determined during the DNS name resolution by network specific protocols.

6 Status

We have implemented an early prototype of *i3* based on the Chord lookup service [15]. We use 256 bit identifiers, and the matching procedure requires exact matching on the 128 most significant bits. This choice makes it very unlikely that a packet will erroneously match a trigger, and at the same time gives the applications up to 128 bits to encode application specific information such as the host location. The untuned prototype is able to forward up to 35,000 packets per second, and to handle about 80,000 trigger insertions/refreshes per second on a 700 MHz Pentium III processor. By choosing the set of Chord fingers based on the network proximity, we are able to reduce the routing latency between any two servers in *i3* within a factor of two (on the average) from the optimal routing in the underlying network.

7 Conclusions

This paper argues for an Internet Indirection Infrastructure based on a rendezvous-based communication abstraction to implement basic communication primitives such as multicast and anycast, and to support end-host mobility and service composition in the Internet. End-hosts no longer communicate by specifying the packet's destination. Instead, they associate with each packet an identifier which is then used by the destination to get the packet. In the best case, we expect this research to open the door to new and exciting research directions in computer networks similar to the way associative memories did in computer architectures, and tuple spaces did in changing the landscape in distributed systems.

References

- [1] N. Carriero. *The Implementation of Tuple Space Machines*. PhD thesis, Yale University, 1987.
- [2] Yatin Chawathe, Steve McCanne, and Eric Brewer. Rmx: Reliable multicast in heterogeneous networks. In *Proceedings of IEEE INFOCOM 2000*, Tel-Aviv, Israel, March 2000.
- [3] D. R. Cheriton and M. Gritter. TRIAD: A new next generation Internet architecture, March 2000. <http://www-dsg.stanford.edu/triad/triad.ps.gz>.
- [4] Y. Chu, S. G. Rao, and H. Zhang. A case for end system multicast. In *Proceedings of ACM SIGMETRICS'00*, pages 1–12, Santa Clara, CA, June 2000.
- [5] Frank Dabek, Frans Kaashoek, David Karger, Robert Morris, and Ion Stoica. Wide-area cooperative storage with cfs. In *Proc. ACM SOSP'01*, pages 202–215, Banff, Canada, 2001.
- [6] S. Deering and D. R. Cheriton. Multicast routing in datagram internetworks and extended LANs. *ACM Transactions on Computer Systems*, May 1990.
- [7] A. Demers, S. Keshav, and S. Shenker. Analysis and simulation of a fair queueing algorithm. In *Journal of Internetworking Research and Experience*, pages 3–26, October 1990.
- [8] P. Francis and R. Gummadi. IPNL: A nat extended internet architecture. In *Proc. ACM SIGCOMM'01*, pages 69–80, San Diego, 2001.
- [9] H.W. Holbrook and D.R. Cheriton. IP multicast channels: EXPRESS support for large-scale single-source applications. In *Proceedings of ACM SIGCOMM'99*, Cambridge, Massachusetts, August 1999.
- [10] Java Spaces. <http://www.javaspaces.homestead.com/>.
- [11] J. Jannotti, D. K. Gifford, K. L. Johnson, M. F. Kaashoek, and Jr. J. W. O'Toole. Overcast: Reliable multicasting with an overlay network. In *Proceedings of the 4th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2000)*, San Diego, California, October 2000.
- [12] P. Mockapetris and K. Dunlap. Development of the Domain Name System. In *Proc. ACM SIGCOMM*, Stanford, CA, 1988.
- [13] A. C. Snoeren and H. Balakrishnan. An end-to-end approach to host mobility. In *Proceedings of ACM/IEEE MOBILECOM'99*, Cambridge, MA, August 1999.
- [14] I. Stoica, T.S.E. Ng, and H. Zhang. REUNITE: A recursive unicast approach to multicast. In *Proceedings of INFOCOM'00*, Tel-Aviv, Israel, March 2000.

- [15] Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. Chord: A scalable peer-to-peer lookup service for internet applications. In *Proc. ACM SIGCOMM'01*, pages 149–160, San Diego, 2001.
- [16] A. Vahdat, M. Dahlin, T. Anderson, and A. Aggarwal. Active names: Flexible location and transport. In *Proceedings of USENIX Symposium on Internet Technologies & Systems*, October 1999.
- [17] W. Adjie-Winoto and E. Schwartz and H. Balakrishnan and J. Lilley. The design and implementation of an intentional naming system. In *Proc. ACM Symposium on Operating Systems Principles*, pages 186–201, Kiawah Island, SC, December 1999.
- [18] P. Wyckoff, S. W. McLaughry, T. J. Lehman, and D. A. Ford. T Spaces. *IBM System Journal*, 37(3):454–474, 1998.